Data Migration Decision Guide

Use this guide to plan a migration to Medplum and decide what moves, which system owns each domain, and how the transition will be validated. Work through identity mapping, import operations, cutover, and exception handling to define an approach with clear acceptance criteria and owners.

Companion to the Migrating to Medplum docs.

Plan the migration around four decisions:

1. **Define scope, identity, and mapping.** Agree on the data and history to move, identifier and deduplication rules, FHIR mappings, and acceptance owners.
2. **Choose loading and validation.** Select an import approach that meets throughput and recovery needs, and define evidence that the migrated data is correct.
3. **Plan cutover and rehearsal.** Decide when write authority changes, how later updates are handled, and what rehearsals must demonstrate.
4. **Plan protection and operational handoff.** Define access and audit controls, preserve usable documents and binaries, and assign monitoring and exception ownership.

Use the context questions to establish the migration's purpose and owners, then check the factors that change the plan for additional design and validation needs.

Before you start

What outcome defines success?

- Replace a legacy clinical or operational system
- Consolidate several systems into one Medplum project
- Backfill history while the source remains active
- Seed selected data for a new workflow
- Preserve a read-only archive
- Build a repeatable onboarding pipeline for future sources

Why: determines whether this is a finite conversion, ongoing synchronization, or a reusable product capability.

Which sources and interfaces are in scope?

For each source, identify:

- Business and technical owners
- Extraction method and available formats
- Data volume and rate of change
- Stable identifiers
- Time zone, locale, and encoding conventions
- Known quality issues and source variants
- Deletion, merge, and correction history
- Whether the source remains active after cutover

Why: a database snapshot, FHIR endpoint, HL7 feed, document archive, and DICOM repository need different extraction and reconciliation plans.

What data and history are required at go-live?

Consider:

- People, organizations, locations, and coverage
- Encounters, appointments, current clinical state, and clinical history
- Forms, communications, tasks, consents, and other workflow state
- Documents, files, and imaging
- Billing, terminology, profiles, and configuration

For each domain, choose full history, current records only, a date range, summary data with legacy archive access, or metadata with files retained elsewhere.

Why: "all data" is not an actionable scope. Each domain needs an explicit boundary, owner, target, and acceptance criteria.

Where is each domain authoritative during and after migration?

- Source remains authoritative
- Medplum becomes authoritative at cutover
- Authority changes in phases
- An external system remains authoritative for a domain
- Both systems coexist under an explicit conflict policy

Why: coexistence without one write authority per domain creates conflicts and makes reconciliation inconclusive.

Who approves the migration?

Assign accountable owners for:

- Source meaning and extraction
- Patient identity
- FHIR mapping and profiles
- Terminology and clinical interpretation
- Security and compliance
- Pipeline operations and reconciliation
- Go-live and rollback

Why: many migration decisions require clinical, operational, and compliance judgment rather than engineering judgment.

What operational change accompanies the migration?

- Which user workflows, reports, and integrations change at cutover?
- How much planned downtime can each workflow tolerate?
- Which users need training, and who owns readiness?
- Who communicates the freeze, cutover, rollback, and return-to-service status?
- What support coverage and post-go-live support period are required?
- How are user-reported data discrepancies triaged after cutover?

Why: technically correct data is not a successful migration if users cannot safely perform their work after go-live.

Factors that change the migration plan

Every migration must define scope, identifiers, mapping ownership, validation, security, testing, cutover authority, and acceptance. The matrix below identifies additional complexity, not optional safeguards.

Use these factors to identify where the migration needs additional design and validation.

Factor	What it changes	Guidance
Multiple source systems, versions, or local variations	Inventory, identifier systems, and mapping layers	Scope and acceptance, FHIR mapping
Cross-source or existing-record patient matching	Identity workflow and reference sequencing	Identity and deduplication
Automated merge execution during migration	Review thresholds, audit, and reversal risk	Identity and deduplication
Custom profiles, extensions, or implementation-guide conformance	Mapping approval and validation configuration	FHIR mapping, Validation

Factor	What it changes	Guidance
One source record producing several linked FHIR resources	Stable IDs, dependency ordering, explicit atomicity requirements, and reconciliation	FHIR mapping, Load architecture
Referenced records arrive after their dependents	Profile eligibility for logical references, pending-link tracking, and a second reconciliation pass	Identity and deduplication, Load architecture
Expected load time may exceed the cutover window or service limits	Import path, throughput testing, and recovery	Load architecture, Dress rehearsals
The source remains active while data moves	Incremental changes, write authority, and conflicts	Write authority
Little or no downtime is acceptable	Change capture, rehearsal depth, and rollback	Write authority, Dress rehearsals
Documents, files, images, or DICOM	Storage, access, integrity, and retrieval testing	Files and complex data
Historical workflows must remain operational	Domain-specific modeling and validation	FHIR mapping, Validation
External systems remaining authoritative after cutover	Long-term ownership and synchronization	Write authority, Operational ownership
Reusable onboarding or ongoing synchronization pipeline	Operational ownership, versioning, and reconciliation	Scope and acceptance, Operational ownership

Cover every topic, adjusting the depth to your migration. Include documents, binaries, and complex data only when those domains are in scope. For each topic, record the decision, accountable owner, and acceptance criteria.

Scope, identity, and mapping

What data and history will move, and who accepts them?

Decide what migrates, how far back each domain goes, and who can accept the result. See Planning your Migration.

Recommendation: Profile the qualifying source population before committing dates, volumes, or validation samples. Give every domain explicit inclusion rules, exclusions, and an acceptance owner.

Questions:

- Which domains and history are required on day one, and what is excluded?
- Which sources contribute to each domain, and can they produce a consistent snapshot or a reliable list of later changes?
- What are the qualifying record counts, file volumes, and change rates?
- Who owns source meaning, mapping, acceptance, and retained legacy access?

Situation	Approach
Small, inactive source	Use a finite extract, transform, load, and reconcile process with a defined freeze window.
Active source	Take a baseline snapshot, then process later changes from a recorded source position until cutover.
Several sources	Keep separate inventories, identifier systems, mappings, and reconciliation results for each source.
Only current state is required	Define “current” separately for each domain; do not apply one generic filter to all resources.
Full history stays in a legacy archive	Define what moves, how older data is retrieved, how long the archive remains, and who owns access.
Scope is uncertain	Profile representative source data before committing dates or sample sizes.

Output: Complete the Source and Domain Inventory.

How will identifiers, matching, and deduplication work?

Decide whether a source record remains distinct, maps to an existing resource, or enters a master-record workflow. See Patient Deduplication Architectures and Patient $match.

⚠ **One-way door:** Loading dependent clinical data before resolving identity can spread references across duplicate patients. Later merges may require extensive repair and clinical review.

Recommendation: Preserve source identifiers using a different identifier system URI for each source. Keep ambiguous matches separate and require human review unless an approved matching policy permits automatic action.

Questions:

- Which source keys are stable and unique, and which identifier system URI represents each source?
- Where can duplicates occur: within a source, across sources, or against Medplum?
- What evidence and risk threshold permit automatic matching versus human review?
- Will migration merge records, or only report candidates for later remediation?
- Can referenced records arrive later, and do profiles and workflows permit those links to remain unresolved temporarily?

Situation	Approach
Stable source key	Preserve it in identifier under a source-specific system URI; use it for writes that are safe to repeat.
Different systems reuse the same value	Use distinct identifier system URIs; never treat the bare value as globally unique.
Source patient may match an existing Patient	Run the approved identity workflow before loading dependent records.
Match is ambiguous	Keep records separate and route to human review.
Existing target ID is not known	Use a conditional reference only when the identifier finds exactly one resource and the migration identity can search it.
Related resources are created together	Load prerequisites first, then batch dependent upserts with saved references; reconcile partial progress.
Related resource was loaded earlier	Use a source-to-target ID lookup or a conditional reference that resolves uniquely.
Related resource will arrive later	If profiles and workflows permit it, preserve a logical reference in Reference.identifier and track the link for a two-pass backfill. Keep actual Patient references for patient-compartment access.
A resolved relationship is required immediately	Load a valid target first or stage the dependent record until the target is available.
Keys are missing or unstable	Define a governed replacement key before loading; do not use mutable demographics as the key for repeated runs.

A source identifier answers “which source record is this?” It does not by itself answer “which real-world person is this?”

Conditional references require exactly one existing match during the write. Logical references can preserve identifiers before a target exists, but the migration must later establish the literal reference. Choose an explicit reconciliation process and acceptance deadline for those links.

How will FHIR mappings and terminology be governed?

Decide how source meaning maps to FHIR R4 and how decisions are versioned and approved. See Governing Data Mappings and Converting Data to FHIR.

⚠ **One-way door:** Changing identifier systems, profile or extension URLs, resource boundaries, or code systems after applications depend on them can require both data and application migrations.

Recommendation: Version mappings independently from pipeline code, preserve source meaning when no verified standard mapping exists, and require domain review before production.

Questions:

- Which FHIR resources, fields, profiles, and terminology represent each source concept?
- Which mappings are direct, conditional, one-to-many, unsupported, or excluded?
- How are missing, ambiguous, free-text, and source-specific values handled?
- Are profiles declared per resource in meta.profile or applied through Project.defaultProfile?
- Who approves changes, and which records must be reprocessed when a mapping changes?

Situation	Approach
Direct mapping	Record source meaning, target path, transformation, null behavior, and validation.
One source entity spans several concepts	Create distinct resources and preserve relationships.
Trustworthy standard code	Preserve its system, code, and display under the approved mapping.
Local terminology only	Preserve a governed local code and text; standard-code enrichment is a separate review.
No verified code mapping	Retain source meaning and route for review; do not invent a code.
No standard field fits	Check established extensions and profiles before defining a custom extension.
One profile applies by resource type	Configure Project.defaultProfile; it applies when the resource has no meta.profile.

Situation	Approach
Profile varies by source record	Set the applicable profile explicitly in meta.profile.
Mapping changes after rehearsal	Version the mapping and code together; define reprocessing or cleanup.

Use the mapping register example to define the information your project must track. Mapping approval is distinct from code review.

Loading and validation

What evidence establishes correctness and acceptance?

Decide how the team will prove structural conformance, referential integrity, semantic correctness, completeness, and workflow readiness. See Validating and Reconciling Migrated Data and Testing and Accepting Migrated Data.

Recommendation: Validate every record structurally and account for every eligible source record. Supplement automated checks with repeatable, random, and risk-based chart review; require named sign-off.

Questions:

- What must reconcile exactly, and which differences are expected and explainable?
- Which relationships, values, and high-risk cohorts require deeper review?
- How is every success, rejection, skip, retry, record held for review, and exception reported?
- Which evidence and thresholds must each stakeholder approve?

Situation	Approach
One source row maps to one resource	Reconcile eligible, successful, rejected, skipped, and held-for-review counts.
One source row maps to several resources	Reconcile each target type and expected relationship separately.
Two-pass backfill leaves pending links	Reconcile pending, resolved, ambiguous, and failed links against stored resources. Verify source identifiers and patient context; require completion or an approved exception before acceptance.
Deduplication changes counts	Report source records, master records, match decisions, and unresolved candidates separately.

Situation	Approach
Profile conformance is required	Confirm the intended profiles are installed and selected and terminology validation is configured, then run $validate; structural validity is not clinical proof.
Clinical meaning cannot be proven by counts	Use repeatable, reproducible random, and risk-based whole-chart samples.
Exceptions remain	Track owner, severity, remediation, acceptance impact, and approval.

Do not prescribe a universal verification percentage. Choose samples after exact extraction counts are known and adjust for variability, risk, complexity, and defect history.

Which load architecture meets throughput and recovery needs?

Decide which writes must succeed or fail together, how the pipeline resumes, and how much throughput the cutover requires. Implementation details and project prerequisites are covered in Building Migration Pipelines.

Recommendation: Use asynchronous batches for high-volume independent writes, and stage related writes in dependency order. Inspect each result and make recovery safe to rerun, observable, and resumable. If a group must commit atomically, resolve that requirement before choosing a hosted load path: atomic transaction Bundles are currently unavailable on hosted Medplum.

Questions:

- Which writes are independent, and which small groups must succeed or fail together?
- Which import path and measured throughput fit the data shape and cutover window?
- How are results, checkpoints, retries, and rejected resources recorded?
- Will pending links be reconciled by a batch second pass or automatically by a Subscription-triggered Bot? Who owns retries, conflicts, and the final sweep?
- Which downstream automations run, and what thresholds pause the load?

For automations that remain active during the load, verify tenant scope and idempotency, and define how missed or failed work is recovered. Apply Planning Automations to these subscriptions; a successful import does not establish that its downstream side effects succeeded.

Pending-link strategy	Choose when	Required recovery
Batch second pass	Related records arrive in known import stages and links can wait until the stage completes.	Keep a durable manifest; retry zero matches after later loads and review ambiguity or concurrent changes.
Subscription-triggered Bot	Targets arrive continuously and links should be completed soon after arrival.	Deploy a custom reconciler, handle either arrival order, make repeated events safe, track handler failures, and run a scheduled or final manifest sweep.

Medplum does not automatically revisit identifier-only references when targets arrive. Both strategies implement the same two-pass reconciliation checks. A Bot can discover dependents in Medplum using reference identifier searches; retain the manifest for recovery and acceptance evidence. Reruns must preserve completed links, and patches must protect concurrent edits with version checks.

Output: Choose a load path, define groups that require all-or-nothing writes and source-of-truth boundaries, and document recovery and side-effect handling in the pipeline design.

Cutover and rehearsal

When does write authority change, and how are later changes handled?

Decide when Medplum becomes authoritative and prevent two systems from independently accepting authoritative writes. See Phased Adoption and Cutover.

⚠ **One-way door:** A cutover is unsafe when two systems accept authoritative writes for the same domain without deterministic synchronization and conflict ownership.

Recommendation: Choose the smallest cutover unit that preserves one clear write authority and can be rehearsed end to end. Use a phased transition only when domains, sites, or cohorts can be separated without ambiguous ownership.

Questions:

- Is transition big bang, phased, parallel, or backfill-only?
- Which system owns reads and writes for each domain and phase?
- How are changes since the previous extract captured, and can any path bypass the authoritative system?
- How are dual-write failures and conflicts handled?
- What evidence triggers go-live, pause, rollback, and legacy retirement?

Situation	Approach
Small, low-change source	Freeze, extract the final changes, load, verify, and switch.
Higher operational risk	Prefer phased transition by domain, site, tenant, or user cohort.
Baseline plus incremental changes	Record the last source change processed and use overlapping windows that are safe to run again.
Temporary dual write	Keep one authoritative path, capture failures durably, and reconcile continuously.
Medplum reads start first	Use feature flags and compare behavior while writes remain in the source.
Legacy becomes an archive	Make it read-only, test access, and disable remaining write integrations.

Output: Complete the Authority and Coexistence Matrix.

What must testing and dress rehearsals demonstrate?

Decide how closely testing must match production volume, infrastructure, access, and workflow behavior. See Testing and Accepting Migrated Data.

Recommendation: Run repeatable mapping tests and rehearse interruption, recovery, reconciliation, access, workflows, and rollback. Use production-scale data and infrastructure when volume, performance, or the cutover window creates material risk.

Questions:

- Which representative data and known edge cases can testing safely use?
- Does the test environment match production features, access, integrations, and scale?
- Have safe reruns, interruption, recovery, throttling, and partial failure been tested?
- Have users rehearsed workflows, documents, cutover, and rollback?

Situation	Approach
Deterministic mapping	Unit test normal, null, malformed, boundary, repeated, and unsupported cases.
Unknown source variation	Profile production-like data and add each discovered defect to regression tests.
Strict cutover window	Rehearse full volume on production-like infrastructure and measure every stage.

Situation	Approach
Pipeline can be interrupted	Stop and resume during rehearsal; verify checkpoints and counts.
Clinical workflows depend on migrated data	Run user acceptance tests with representative patient journeys.
Access varies by role or tenant	Test allowed and denied access using actual migrated references and compartments.

A small pilot does not prove production throughput. A full-volume load does not prove clinical meaning. Test both when those risks apply.

Protection and operational handoff

How will migration data, credentials, and audit evidence be protected?

Decide how protected data is extracted, staged, transmitted, logged, retained, and destroyed. See Access Policies and HIPAA Compliance.

Recommendation: Use dedicated least-privilege credentials, approved encrypted storage, PHI-minimized logs, and explicit retention and disposal rules for every temporary artifact.

Questions:

- Which people, systems, and environments may access migration PHI?
- How are extracts, credentials, logs, rejected records, and reports protected?
- What audit, retention, revocation, and secure-disposal rules apply?
- What incident process handles a wrong-patient, wrong-tenant, or wrong-project load?

Situation	Approach
Production automation	Use a dedicated ClientApplication and narrowly scoped AccessPolicy, not a personal token.
Extracts contain PHI	Use approved encrypted storage, restrict access, and destroy temporary copies after acceptance.
Logs contain source data	Minimize fields and apply regulated-data access and retention controls.
Production-like test data is needed	Prefer synthetic or de-identified data unless approved controls authorize PHI.
Multi-tenant target	Define the tenant AccessPolicy; populate each resource's own patient references and/or governed meta.account; test allowed and denied access.

Situation	Approach
Wrong-scope load	Stop the pipeline, preserve evidence, assess exposure, and use a reviewed correction plan.

Security review covers the pipeline and temporary artifacts, not only final FHIR resources.

How will documents, binaries, and complex data remain usable?

Decide whether complex data becomes structured resources, binary content, metadata, external references, or a separate workstream.

Recommendation: Treat each complex domain as its own scoped workstream. Do not load it until its FHIR model, content storage, access rules, integrity checks, and retrieval workflow are approved.

Questions:

- Which complex domains are in scope, and must they remain operational or only viewable?
- Are original bytes, structured data, searchable metadata, or all three required?
- How are identity, clinical context, access, integrity, and retrieval verified?
- Will files use SDK uploads, presigned upload/download URLs, or source URLs consumed by auto-download? Who renews expired URLs and resumes failed transfers?
- Which unsupported or ambiguous data needs human review or legacy retention?

Use Binary Data, the Messaging Data Model, and the DICOM Data Model rather than defining those models inside the migration plan.

For files, choose the transfer path separately from the metadata model:

Situation	Approach
The migration process already has the bytes and Medplum credentials	Use the SDK file helpers with the intended security context and patient-linked metadata.
A separate process needs temporary upload access	Create the Binary metadata with its security context, then obtain a presigned upload URL with $presigned-url?upload=true. Upload bytes with PUT, track the Binary ID for retries, and verify completion.
A process needs temporary download access	Obtain a fresh presigned download URL through $presigned-url or a returned attachment URL; use GET and verify the bytes.

Situation	Approach
The source supplies accessible file URLs	Consider auto-download; source presigned URLs must remain valid through queue delays and retrieval retries. Track failures and renew expired source URLs.

See Uploading and Downloading with Presigned URLs for the request flow. Store stable Binary/{id} references in metadata and manifests. Presigned URLs expire and grant access to their holders, so exclude them from logs and durable manifests. Define byte-integrity checks and a completion gate that distinguishes metadata creation from a successful file transfer.

Finalize Binary metadata before requesting an upload URL and keep it unchanged until the upload is verified. Track the Binary version for retries; if it changes, stop old transfers and obtain a fresh URL. A presigned upload overwrites bytes without a new FHIR version or history entry. If earlier content must be retained, upload replacements to a new Binary before changing the attachment.

Who monitors runs, resolves exceptions, and accepts operational ownership?

Decide how production runs are controlled and when migration ownership transfers to normal operations.

Recommendation: Assign service ownership before production. Keep exceptions visible through resolution, define automatic pause thresholds, and close the migration only after operational handoff and temporary-access cleanup.

Questions:

- Which metrics, alerts, and thresholds pause a run?
- Who owns technical, identity, clinical, and terminology exceptions?
- Does the pipeline end or become an ongoing production integration?
- What ends the post-go-live support period and permits credential, staging, and source-access retirement?

Situation	Approach
Finite migration	Define completion, archive evidence, revoke credentials, and dispose of temporary data.
Ongoing synchronization	Treat it as a production integration with service ownership, alerts, runbooks, and reconciliation cadence.
Transient failure	Retry with bounded backoff from a durable checkpoint.

Situation	Approach
Repeatable mapping or data failure	Hold the record for review, fix the mapping or data, add a regression test, and safely rerun affected records.
Identity ambiguity	Route to the identity-review owner, not a generic retry queue.
Clinical ambiguity	Route to the clinical or terminology owner.
Error threshold exceeded	Pause the affected stream, preserve state, and resume only after approval.
Users need new workflows	Assign training, communications, support, and readiness owners before cutover.

Before You Implement

Record:

- Data scope, history boundaries, and exclusions
- Identifier, matching, and mapping decisions
- Load path, recovery approach, and operational owner
- Pending-link strategy, reconciliation owner, and completion criteria when referenced data arrives later
- File transfer method, URL renewal, and byte-verification criteria when files are in scope
- Write authority, cutover approach, and rollback criteria
- Validation evidence, acceptance thresholds, and approvers
- Security controls and unresolved risks

Do not begin production migration work while any decision that changes identity, clinical meaning, write authority, or acceptance remains unowned or unresolved.